

Lab 4 – Endless Runner: LLM chat history (Claude Code, Opus 5.5)

```
■ ■ ■ ■ Claude Code v2.1.285
■ ■ ■ ■ Opus 5.5 · Claude Max
■ ■ ■ ■ ~/sem5/SE/lab4/39_endless-runner
```

```
> /clear
```

```
> complete this: https://github.com/SETAPESU26/39\_endless-runner
```

Ran 4 shell commands (ctrl+o to expand)

- Write(game/sound.py)

```
└ Wrote 46 lines to game/sound.py
  1 import math
  2 from array import array
  3
  4
  5
  6 # Sound effects are synthesized at startup so the project needs
  7   no audio
  8 # asset files. If no audio device is available, every sound beco
  9
 10 class _Silent:
... +42 lines (ctrl+o to expand)
```

- Sound module done; now rewriting the engine for swept collision, speed cap, game-over screen, and difficulty replay.

- Bash(python3 - <<'EOF'

```
  p='game/sound.py'...)
└ Updated game/player.py (+2 -0)
  16         if self.on_ground:
  17             self.vy = self.jump_strength
  18             self.on_ground = False
  19 +             return True
  20 +             return False
  21
  22         def update(self):
  23             self.vy += self.gravity
└ Updated game/sound.py (+4 -1)
  37 class SoundBank:
  38     def __init__(self):
  39         try:
  40 -             if not pygame.mixer.get_init():
  40 +             init = pygame.mixer.get_init()
  41 +             if not init or init[1] != -16:
  42 +                 # Samples are built as signed 16-bit, so force
  43 +                 +that format.
  43 +                 pygame.mixer.quit()
  44                 pygame.mixer.init(frequency=44100, size=-16, ch
  45 annels=1)
  45                 self.jump = _tone([400, 800], 0.12)
  46                 self.score = _tone([880, 1320], 0.08, volume=0.3)
```

- Write(game/game_engine.py)

```
└ Added 73 lines, removed 24 lines
  1 import pygame
  2 from .player import Player
  3 from .obstacle import Obstacle
  4 +from .sound import SoundBank
  5
  6 # Game Engine
  7
```

```

8 WHITE = (255, 255, 255)
9 +BLACK = (0, 0, 0)
10 BROWN = (120, 80, 40)
11 DARK_GREEN = (30, 100, 30)
12
13 +# Starting speed, frames between spawns, and the speed ceiling
  +per difficulty.
14 +DIFFICULTIES = {
15 +     "Easy": {"speed": 5, "spawn_interval": 90, "max_speed": 1
  +2},
16 +     "Medium": {"speed": 6, "spawn_interval": 70, "max_speed": 1
  +5},
17 +     "Hard": {"speed": 8, "spawn_interval": 55, "max_speed": 1
  +8},
18 +}
19 +
20 +DIFFICULTY_KEYS = {
21 +     pygame.K_1: "Easy", pygame.K_e: "Easy",
22 +     pygame.K_2: "Medium", pygame.K_m: "Medium",
23 +     pygame.K_3: "Hard", pygame.K_h: "Hard",
24 +}
25 +
26 class GameEngine:
27     def __init__(self, width, height):
28         self.width = width
29         self.height = height
30         self.ground_y = height - 40
31
32 +         self.font = pygame.font.SysFont("Arial", 30)
33 +         self.big_font = pygame.font.SysFont("Arial", 56, bold=T
+true)
34 +         self.small_font = pygame.font.SysFont("Arial", 22)
35 +         self.sounds = SoundBank()
36 +         self.quit_requested = False
37 +
38 +         self.reset("Medium")
39 +
40 +     def reset(self, difficulty):
41 +         settings = DIFFICULTIES[difficulty]
42 +         self.difficulty = difficulty
43 +
44 +         self.player = Player(80, self.ground_y)
45
46 -         self.speed = 6
47 -         # NOTE: speed keeps climbing every frame with no ceilin
-g. See
48 -         # Task 1 in the README for why that eventually breaks c
-ollision.
49 +         self.speed = settings["speed"]
50 +         self.speed_increase_per_frame = 0.003
51 +         # Ceiling on speed so the game stays playable on long r
+uns.
52 +         self.max_speed = settings["max_speed"]
53
54 -         self.spawn_interval = 70 # frames between obstacle spa
-wns
55 +         self.spawn_interval = settings["spawn_interval"] # fra
+mes between obstacle spawns
56 +         self._spawn_timer = 0
57 +         self.obstacles = []
58
59 +         self.distance = 0
60 +         self.score = 0
61 -         self.font = pygame.font.SysFont("Arial", 30)
62 -         self.game_over = False
63
64     def handle_event(self, event):

```

```

34 -         if event.type == pygame.KEYDOWN and event.key in (pygame
-e.K_SPACE, pygame.K_UP, pygame.K_w):
35 -             self.player.jump()
60 +         if event.type != pygame.KEYDOWN:
61 +             return
62
63 +         if self.game_over:
64 +             if event.key in DIFFICULTY_KEYS:
65 +                 self.reset(DIFFICULTY_KEYS[event.key])
66 +             elif event.key in (pygame.K_ESCAPE, pygame.K_q):
67 +                 self.quit_requested = True
68 +             return
69 +
70 +         if event.key in (pygame.K_SPACE, pygame.K_UP, pygame.K_
+w):
71 +             if self.player.jump():
72 +                 self.sounds.jump.play()
73 +
74     def handle_input(self):
75         # Reserved for continuously-held-key input; this runner
only
76         # needs an edge-triggered jump, handled in handle_event
.
...
80         if self.game_over:
81             return
82
46 -         self.speed += self.speed_increase_per_frame
83 +         self.speed = min(self.speed + self.speed_increase_per_f
+frame, self.max_speed)
84         self.player.update()
85
86         self._spawn_timer += 1
...
89         self.obstacles.append(Obstacle(self.width, self.gr
ound_y, self.speed))
90
91         for obstacle in self.obstacles:
55 -             obstacle.move()
92             obstacle.speed = self.speed
93 +             obstacle.move()
94
58 -         # NOTE: collision is only checked against each obstacl
-e's rect
59 -         # *after* it has already moved for the frame. There's
-no check
60 -         # for whether the obstacle's path crossed the player a
-long the
61 -         # way. Since self.speed has no cap, it eventually grow
-s large
62 -         # enough that an obstacle can cover more horizontal di
-stance
63 -         # in one frame than the player's own width - meaning i
-t can be
64 -         # entirely to the right of the player on one frame and
-entirely
65 -         # to the left on the next, skipping the player's hitbo
-x
66 -         # completely and never registering a hit. See Task 1.
95 +         # Swept collision: test the full horizontal span each
+obstacle
96 +         # covered this frame, so a fast obstacle can't jump pa
+st the
97 +         # player between two frames without registering a hit.
98 +         player_rect = self.player.rect()
99         for obstacle in self.obstacles:
68 -             if obstacle.rect().colliderect(self.player.rect())

```

```

-:
100 +         if obstacle.swept_rect().collidect(player_rect):
101             self.game_over = True
102 +         self.sounds.game_over.play()
103             return
104
105         for obstacle in self.obstacles:
106             if not obstacle.scored and obstacle.x + obstacle.w
idth < self.player.x:
107                 obstacle.scored = True
108                 self.score += 1
109 +                 self.sounds.score.play()
110
111             self.obstacles = [o for o in self.obstacles if not o.o
ff_screen()]
112
...
119         for obstacle in self.obstacles:
120             pygame.draw.rect(screen, DARK_GREEN, obstacle.rect
())
121
88 -         score_text = self.font.render(f"Score: {self.score}",
-True, (0, 0, 0))
122 +         score_text = self.font.render(f"Score: {self.score}",
+True, BLACK)
123             screen.blit(score_text, (10, 10))
124 +         diff_text = self.small_font.render(self.difficulty, Tr
+ue, BLACK)
125 +         screen.blit(diff_text, (self.width - diff_text.get_wid
+th() - 10, 14))
126
91 -         if self.game_over and not getattr(self, "_game_over_lo
-gged", False):
92 -             # NOTE: no proper game-over screen yet - see Task
-2 in the README.
93 -             print("Game over! Final score:", self.score)
94 -             self._game_over_logged = True
127 +         if self.game_over:
128 +             self._render_game_over(screen)
129 +
130 +         def _render_game_over(self, screen):
131 +             overlay = pygame.Surface((self.width, self.height), py
+game.SRCALPHA)
132 +             overlay.fill((0, 0, 0, 170))
133 +             screen.blit(overlay, (0, 0))
134 +
135 +             lines = [
136 +                 (self.big_font, "GAME OVER", 110),
137 +                 (self.font, f"Final score: {self.score}", 175),
138 +                 (self.small_font, "Play again: 1 Easy 2 Medium
+ 3 Hard", 240),
139 +                 (self.small_font, "Esc / Q to quit", 275),
140 +             ]
141 +             for font, text, y in lines:
142 +                 surf = font.render(text, True, WHITE)
143 +                 screen.blit(surf, surf.get_rect(center=(self.width
+ // 2, y)))

```

- Bash(python3 - <<'EOF'
 - p='game/obstacle.py'...)
 - └ Created .gitignore (+2 -0)
 - 1 +__pycache__/
 - 2 +.venv/
 - └ Updated game/obstacle.py (+8 -0)
 - 8 self.y = ground_y - height
 - 9 self.speed = speed
 - 10 self.scored = False

```

11 +         self.prev_x = x
12
13     def move(self):
14 +         self.prev_x = self.x
15         self.x -= self.speed
16
17     def off_screen(self):
...
19
20     def rect(self):
21         return pygame.Rect(self.x, self.y, self.width, self.height)
22 +
23 +     def swept_rect(self):
24 +         """Rect covering everywhere the obstacle occupied during
25 +         its last move."""
26 +         left = min(self.x, self.prev_x)
27 +         right = max(self.x, self.prev_x) + self.width
28 +         return pygame.Rect(left, self.y, right - left, self.height)
29
30
31 [ Updated main.py (+2 -0)
32
33     if event.type == pygame.QUIT:
34         running = False
35     engine.handle_event(event)
36
37     if engine.quit_requested:
38         running = False
39
40
41     engine.handle_input()
42     engine.update()

```

Read 1 file, ran 1 shell command (ctrl+o to expand)

- I finished all four tasks in a fresh clone at lab4/39_endless-runner. A headless test run passed on every change. I haven't played it with a real window and audio, and I haven't committed or pushed anything.
- Task 1, collision:
 - Speed now has a top limit: Easy 12, Medium 15, Hard 18.
 - Hits are now checked across the whole distance an obstacle moved during the frame, not just where it ends up (in game/obstacle.py).
 - Test: an obstacle moving 60px per frame slipped past the player under the old check. The new check catches it.
- Task 2, game over screen: A dimmed overlay shows "GAME OVER" and the final score, and the game waits for a key. This replaces the old console print.
- Task 3, replay: On the game over screen, press 1/E for Easy, 2/M for Medium or 3/H for Hard, or Esc/Q to quit. Space is ignored there, so a held jump key can't restart the game by accident. Each level has its own starting speed and time between obstacles. The first game starts on Medium.
- Task 4, sound: New file game/sound.py makes the jump, score and game-over sounds in code, so there are no audio files to add. If there's no audio device, it stays silent instead of crashing. The jump sound plays only when a jump actually happens.
- Other changes:
 - Each obstacle now gets its new speed before it moves, not after.
 - main.py exits when you choose quit.
 - A .gitignore covers __pycache__/ and .venv/.

To run it: .venv/bin/python main.py. pygame is installed in .venv.

For your "before" video, check out the original code with git stash, record it, then run git stash pop. The new game/sound.py and .gitignore aren't tracked, so they stay in place during the stash. That doesn't affect the old version, because nothing in the original code uses them.

* Worked for 1m 41s · done 11:55 AM

* recap: All four endless-runner lab tasks are done in lab4/39_endless-runner and pass a headless

test: collision fix, game-over screen, difficulty replay and sounds. Next, play it with
'`.venv/bin/python main.py`' and record your before and after videos. (disable recaps in `/config`)

> also can you take the video and save it too

Made 1 scratchpad edit +130, ran 4 shell commands (ctrl+o to expand)

- Rendering the before/after clips headlessly and encoding with ffmpeg.

Ran 1 shell command (ctrl+o to expand)

- Script dir is on `sys.path`, not `cwd`; adding `cwd` via `PYTHONPATH`.

Read 2 files, ran 3 shell commands (ctrl+o to expand)

- I made both 10-second videos and saved them in `lab4/videos/`. They're outside the repo, so they won't get committed.

- before.mp4 (original code):

- Speed is set to 75 px/frame, where it would be about 6 minutes into a normal run, so the bug shows up within 10 seconds. The caption on the video says this.
- The player never jumps, yet 6 obstacles pass straight through and the score still goes up.
- At about 8.3 seconds one obstacle finally hits. The game just freezes, with no game over screen. The only sign is "Game over!" printed to the console.
- It has no sound, because the original code has none.

- after.mp4 (your fixed code):

- It plays on Medium with speed held at the 15 px/frame cap, jumping over obstacles with the jump and score sounds.
- Then it stops jumping, hits an obstacle and shows the game over screen with the final score, plus the game-over sound.
- Then key 3 is pressed and a new game starts on Hard.
- Captions show the speed, and the keys pressed appear on screen.

These aren't screen captures of a real window. A script played the game automatically and drew each frame straight to video, then added the sound. It runs your actual game code, but a computer controls the player. If your instructor wants a real screen recording, you can record one with `Cmd+Shift+5`.

At 75 px/frame the obstacles in the "before" video only show for a few frames each, so they flicker past. That speed is the bug, but it's worth knowing before you submit.

The script that made the videos is in this session's temporary folder, so it won't stay around. Say if you want it copied into `lab4/`.

* Cooked for 1m 58s · done 12:02 PM

> Instructions:

1. Go to the repo assigned to you
 2. Go through the ReadMe file to understand the deliverables
 3. Clone or Fork the repo
 4. Run the Python based code (A game/ A Puzzle)
 5. Record 10 seconds of video before making any changes.
 6. Write prompts and fix the broken code
 7. Write prompts to add all the features listed in the ReadMe file of the repo one by one.
 8. Record another 10 seconds of video again after making all the changes.
 9. Push the code to your repo.
 10. Do not raise PR to the main repo (SETAPESU26)
- Deliverables:
- Upload the following to your Lab repo under Lab-4 folder,
- a. Videos (before and after)
 - b. Updated Code
 - c. Chat history exported as a doc/pdf

Ran 2 shell commands (ctrl+o to expand)

- User answered Claude's questions:

└ · Which repo is your Lab repo to upload Lab-4 into? → maybe we should fork it ig?

Ran 1 shell command (ctrl+o to expand)

- Bash(git remote rename origin upstream && git remote add origin git@github.com:Gagancreates/39_endless-runner.git && git remote set-url --push upstream DISABLED && g...)
└ origin git@github.com:Gagancreates/39_endless-runner.git (fetch)
origin git@github.com:Gagancreates/39_endless-runner.git (push)
upstream https://github.com/SETAPESU26/39_endless-runner (fetch)
... +12 lines (ctrl+o to expand)

└ 2 files changed (binary, mode only or too large to show)

Pushed to main, ran 1 shell command (ctrl+o to expand)

- I forked the repo and pushed the finished code and both videos to your fork: https://github.com/Gagancreates/39_endless-runner. I didn't open a PR to SETAPESU26, and I switched off pushing to their copy of the repo on your machine so it can't happen by accident.

- Updated code: at the top level of the fork, so the game still runs with python main.py.
- Videos: Lab-4/before.mp4 and Lab-4/after.mp4.
- Cleanup: the committed __pycache__ files are removed from the repo, and .gitignore now keeps them and .env/ out.

Still to do:

- Chat history: I can't export this conversation myself. Run /export to save it as a text file, and I'll convert it to a PDF and add it to Lab-4/. Or use the claude.ai share link if a link is acceptable.
- Lab-4 location: I put Lab-4/ inside the fork. If your class wants one lab repo that holds every lab's folder, I can copy the code in there too.
- Videos: these are the script-rendered clips from earlier, not screen recordings. If your instructor wants a real recording, swap them in Lab-4/.

* Brewed for 46s · done 12:08 PM

> /export

└ Conversation copied to clipboard

> this is the chat history, remove the things where er mrntioned about rendered screen recordings

```
■ ■ ■ Claude Code v2.1.285
    ■ Opus 5.5 · Claude Max
■ ■ ~ /sem5/SE/lab4/39_endless-runner
```

> /clear

> complete this: https://github.com/SETAPESU26/39_endless-runner

Ran 4 shell commands (ctrl+o to expand)

- Write(game/sound.py)

└ Wrote 46 lines to game/sound.py
1 import math
2 from array import array
3

```

5
6 # Sound effects are synthesized at startup so the project needs
  no audio
7 # asset files. If no audio device is available, every sound beco
8
9
10 class _Silent:
... +42 lines (ctrl+o to expand)

```

- Sound module done; now rewriting the engine for swept collision, speed cap, game-over screen, and difficulty replay.

- Bash(`python3 - <<'EOF'`
`p='game/sound.py'...`)
 - └ Updated game/player.py (+2 -0)


```

16         if self.on_ground:
17             self.vy = self.jump_strength
18             self.on_ground = False
19 +             return True
20 +             return False
21
22     def update(self):
23         self.vy += self.gravity
          
```
 - └ Updated game/sound.py (+4 -1)


```

37 class SoundBank:
38     def __init__(self):
39         try:
40 -             if not pygame.mixer.get_init():
40 +             init = pygame.mixer.get_init()
41 +             if not init or init[1] != -16:
42 +                 # Samples are built as signed 16-bit, so force
+that format.
43 +                 pygame.mixer.quit()
44                 pygame.mixer.init(frequency=44100, size=-16, ch
annels=1)
45                 self.jump = _tone([400, 800], 0.12)
46                 self.score = _tone([880, 1320], 0.08, volume=0.3)
          
```

- Write(game/game_engine.py)
 - └ Added 73 lines, removed 24 lines


```

1 import pygame
2 from .player import Player
3 from .obstacle import Obstacle
4 +from .sound import SoundBank
5
6 # Game Engine
7
8 WHITE = (255, 255, 255)
9 +BLACK = (
          
```

(324 lines hidden)

of video again after making all the changes.

9. Push the code to your repo.

10. Do not raise PR to the main repo (SETAPESU26)

Deliverables:

Upload the following to your Lab repo under Lab-4 folder,

- Videos (before and after)
- Updated Code
- Chat history exported as a doc/pdf

Ran 2 shell commands (ctrl+o to expand)

- User answered Claude's questions:

└ · Which repo is your Lab repo to upload Lab-4 into? → maybe we should fork it ig?

Ran 1 shell command (ctrl+o to expand)

- Bash(git remote rename origin upstream && git remote add origin git@github.com:Gagancreates/39_endless-runner.git && git remote set-url --push upstream DISABLED && g...)
└ origin git@github.com:Gagancreates/39_endless-runner.git (fetch)
origin git@github.com:Gagancreates/39_endless-runner.git (push)
upstream https://github.com/SETAPESU26/39_endless-runner (fetch)
... +12 lines (ctrl+o to expand)

└ 2 files changed (binary, mode only or too large to show)

Pushed to main, ran 1 shell command (ctrl+o to expand)

- I forked the repo and pushed the finished code and both videos to your fork:

https://github.com/Gagancreates/39_endless-runner. I didn't open a PR to SETAPESU26, and I switched off pushing to their copy of the repo on your machine so it can't happen by accident.

- Updated code: at the top level of the fork, so the game still runs with python main.py.
- Videos: Lab-4/before.mp4 and Lab-4/after.mp4.
- Cleanup: the committed __pycache__ files are removed from the repo, and .gitignore now keeps them and .venv/ out.

Still to do:

- Chat history: I can't export this conversation myself. Run /export to save it as a text file, and I'll convert it to a PDF and add it to Lab-4/. Or use the claude.ai share link if a link is acceptable.
- Lab-4 location: I put Lab-4/ inside the fork. If your class wants one lab repo that holds every lab's folder, I can copy the code in there too.
- Videos: these are the script-rendered clips from earlier, not screen recordings. If your instructor wants a real recording, swap them in Lab-4/.

* Brewed for 46s · done 12:08 PM

- I won't cut those parts. The README asks for the complete chat history, and deleting the lines about how the videos were made would hide that from your instructor. That's the part I'm not comfortable editing out.

There's an honest way to make the issue go away: replace the videos with real screen recordings, so nothing in the chat needs hiding.

- I record the real game window: I launch the game in a normal window and capture the screen with ffmpeg, once on the original code and once on the fixed code. macOS will ask you to grant Screen Recording permission to your terminal the first time. The before clip would still start at high speed so the bug shows in 10 seconds, and I'd note that in the video.
- You record it yourself: run .venv/bin/python main.py and press Cmd+Shift+5, once on the original code (git stash, record, git stash pop) and once on the fixed code. I'll swap the new files into Lab-4/ and push.

Either way, I'll turn the full export into a PDF, add it to Lab-4/ and push it. Start with the first option if you want it done without playing yourself.

* Sautéed for 11s · done 12:13 PM

> ok

Made 1 scratchpad edit +46, read 1 file, ran 2 shell commands (ctrl+o to expand)

- Screen capture works. Recording the original game in a real window now.

Read 1 file, ran 4 shell commands (ctrl+o to expand)

- User answered Claude's questions:

└ · My first capture recorded your browser because the game window opened behind it. I deleted that clip. The next run brings the game window to the front for about 15 seconds, twice. Can you leave the Mac alone (no clicking or switching apps) for about 45 seconds? → Go ahead

Read 2 files, ran 4 shell commands (ctrl+o to expand)

- Both real window recordings captured; trimming to 10s and cropping to the game window.

Read 1 file, ran 3 shell commands (ctrl+o to expand)

- I recorded the game running in a real window on your screen, replaced both videos in Lab-4/, and pushed them to your fork.
 - before.mp4 (original code): the speed starts at 75 px/frame. At the normal ramp it would take about 6 minutes to get that fast, so I set it directly. The player never jumps, yet 6 obstacles pass through and the score still rises. One obstacle finally hits and the game just freezes, with no game over screen.
 - after.mp4 (your fixed code): it plays on Medium and jumps over obstacles. Then it stops jumping and the game over screen appears with the final score. Then 3 is pressed and a new game starts on Hard.
 - Input: a small script pressed the keys; nobody played by hand.
 - Sound: the audio came from your Mac's microphone, since macOS can't record system sound directly. I boosted it, but the game sounds are still faint and there's some room noise.
 - Mouse pointer: your pointer shows in the corner of the after clip.
 - Cleanup: I deleted the raw full-screen captures, which also showed your WhatsApp window. lab4/videos/ now holds copies of the new clips.

For the chat history, run /export lab4-chat.txt now so it includes this last part too. I'll turn it into a PDF, add it to Lab-4/ and push it.

* Brewed for 3m 15s · done 12:16 PM